Análisis Experimental de la Complejidad Temporal del Algoritmo Selection Sort frente a Merge Sort

Experimental Analysis of the Time Complexity of the Selection Sort Algorithm Compared to Merge Sort

Joshua David Huerta Choque
Escuela Profesional de Ingeniería en Informática y Sistemas
Universidad Nacional Jorge Basadre Grohmann
Tacna, Perú
ORCID: https://orcid.org/0009-0009-9181-2648
Correo institucional: jdhuertac@estudiante.unjbg.edu.pe

Resumen— El objetivo de esta investigación fue evaluar el rendimiento empírico del algoritmo Selection Sort frente a métodos de ordenamiento eficientes. Se implementó una suite de pruebas automatizadas en C++17 para medir tiempos de ejecución, comparaciones e intercambios sobre arreglos de $n = 1,000$ y $n = 10,000$ elementos bajo cuatro patrones de datos y 30 repeticiones independientes. Los resultados mostraron que Selection Sort mantuvo un comportamiento cuadrático invariable ($\approx 49.9 \times 10^6$ comparaciones para $n = 10,000$), requiriendo tiempos significativamente superiores a las alternativas $O(n \log n)$. Se concluye que Selection Sort carece de adaptabilidad y que la omisión de $n = 100,000$ está justificada por la degradación exponencial del rendimiento.

Palabras clave— Selection Sort, complejidad computacional, análisis empírico, algoritmos de ordenamiento, Merge Sort, C++.

Abstract— The objective of this research was to evaluate the empirical performance of the Selection Sort algorithm compared to efficient sorting methods. An automated test suite in C++17 was implemented to measure execution times, comparisons, and swaps on arrays of $n = 1,000$ and $n = 10,000$ elements under four data patterns and 30 independent repetitions. Results showed that Selection Sort maintained an invariant quadratic behavior ($\approx 49.9 \times 10^6$ comparisons for $n = 10,000$), requiring significantly higher execution times than $O(n \log n)$ alternatives. It is concluded that Selection Sort lacks adaptability and that omitting $n = 100,000$ is justified by exponential performance degradation.

Keywords— Selection Sort, computational complexity, empirical analysis, sorting algorithms, Merge Sort, C++.

1. Introducción

El análisis empírico de algoritmos permite contrastar los modelos teóricos de complejidad asintótica (notación Big-O) con el rendimiento real en entornos de ejecución modernos. En la enseñanza de la computación, el algoritmo directo Selection Sort se estudia ampliamente debido a su sencillez de diseño; sin embargo, exhibe una cota de rendimiento de orden cuadrático $O(n^2)$, lo que limita severamente su escalabilidad. Como contraste fundamental, los enfoques eficientes basados en la técnica de divide y vencerás, como MergeSort, garantizan una complejidad temporal de $O(n \log n)$ y estabilidad en el procesamiento de datos.

El objetivo de este trabajo es caracterizar experimentalmente la latencia, el consumo de operaciones y la estabilidad del algoritmo Selection Sort en comparación directa con las ventajas de rendimiento de MergeSort. Se plantea como hipótesis descriptiva que el comportamiento de Selection Sort se mantiene rígido e inalterable frente a la disposición inicial de las entradas, mientras que MergeSort ofrece un rendimiento óptimo, predecible y escalable ante diversos volúmenes de información.

2. Método

2.1. Algoritmos y Complejidad Teórica

Se evaluó el algoritmo Selection Sort, cuya complejidad teórica en el peor, promedio y mejor caso es de $O(n^2)$ para comparaciones, realizando $O(n)$ intercambios debido a su naturaleza iterativa basada en la búsqueda secuencial del mínimo. Como contraparte eficiente, se implementó MergeSort, un algoritmo basado en la técnica de divide y vencerás que garantiza una complejidad temporal de $O(n \log n)$ en todos los escenarios (mejor, promedio y peor). A diferencia de Selección, MergeSort es un algoritmo estable, aunque requiere espacio de memoria adicional auxiliar de $O(n)$ para la mezcla de los subarreglos.

2.2. Entorno de Pruebas

Las pruebas se ejecutaron en una máquina con sistema operativo Windows 11, compilador `g++` con estándar C++17 y la bandera de optimización `-O2`. Para la toma de tiempos se empleó la librería `std::chrono::steady_clock` de precisión en nanosegundos.

2.3. Generación de Datos y Repeticiones

Se configuraron tamaños de entrada $n = 1,000$ y $n = 10,000$ elementos. Se omitió formalmente $n = 100,000$ debido al tiempo inviable de ejecución impuesto por la complejidad

cuadrática. Se evaluaron cinco patrones de datos (Aleatorio, Ascendente, Descendente, Casi ordenado y Con duplicados) utilizando el generador std::mt19937 con semillas fijas para asegurar reproducibilidad. Para cada combinación se ejecutaron 30 repeticiones independientes.

3. Resultados

Los datos obtenidos de las 30 repeticiones independientes almacenados en el archivo ‘results/resultados_experimento.csv‘ se procesaron para evaluar el comportamiento empírico de ambos algoritmos. La Figura 1 detalla la estadística descriptiva (media, mediana, desviación estándar, valores mínimos y máximos) de los tiempos de ejecución, confirmando la estabilidad de las mediciones experimentales.

=== TABLA DE ESTADÍSTICA DESCRIPTIVA (TIEMPO EN MS) ===

	algoritmo	n	patron	Media	Mediana	Desv_Est	Minimo	Maximo
0	MergeSort	1000	Aleatorio	0.450940	0.43835	0.036472	0.4262	0.6219
1	MergeSort	1000	Ascendente	0.364680	0.36335	0.004120	0.3601	0.3777
2	MergeSort	1000	CasiOrdenado	0.399437	0.39410	0.017967	0.3894	0.4623
3	MergeSort	1000	Descendente	0.370603	0.36525	0.021789	0.3614	0.4811
4	MergeSort	1000	Duplicados	0.419253	0.41910	0.002721	0.4134	0.4255
5	MergeSort	10000	Aleatorio	5.273613	5.24620	0.112704	5.2218	5.8513
6	MergeSort	10000	Ascendente	4.418720	4.35850	0.206581	4.3000	5.2949
7	MergeSort	10000	CasiOrdenado	5.055090	4.83515	0.565464	4.6956	7.2315
8	MergeSort	10000	Descendente	4.462930	4.30930	0.347796	4.2490	5.9451
9	MergeSort	10000	Duplicados	6.622780	5.04450	7.852333	4.9327	48.1102
10	SelectionSort	1000	Aleatorio	3.829700	3.68660	0.338597	3.5988	4.9371
11	SelectionSort	1000	Ascendente	3.577523	3.56555	0.061055	3.5428	3.8764
12	SelectionSort	1000	CasiOrdenado	3.668573	3.61265	0.196100	3.5853	4.4553
13	SelectionSort	1000	Descendente	3.755507	3.64220	0.351648	3.5940	5.1608
14	SelectionSort	1000	Duplicados	3.617720	3.61370	0.030388	3.5773	3.6925
15	SelectionSort	10000	Aleatorio	360.734400	358.59700	6.206786	356.4200	380.8840
16	SelectionSort	10000	Ascendente	370.741633	365.13400	20.246614	355.8610	462.5390
17	SelectionSort	10000	CasiOrdenado	388.915300	373.15550	46.790423	358.0120	612.5040
18	SelectionSort	10000	Descendente	374.300633	369.69250	13.378141	361.3650	430.5680
19	SelectionSort	10000	Duplicados	372.382433	365.06050	24.064405	357.3170	481.6590

Figura 1: Tabla detallada de la estadística descriptiva

Asimismo, la Figura 2 ilustra la comparación directa de los tiempos promedio de ejecución por tamaño de entrada, evidenciando el crecimiento exponencialmente superior de Selection Sort frente a la estabilidad de MergeSort.

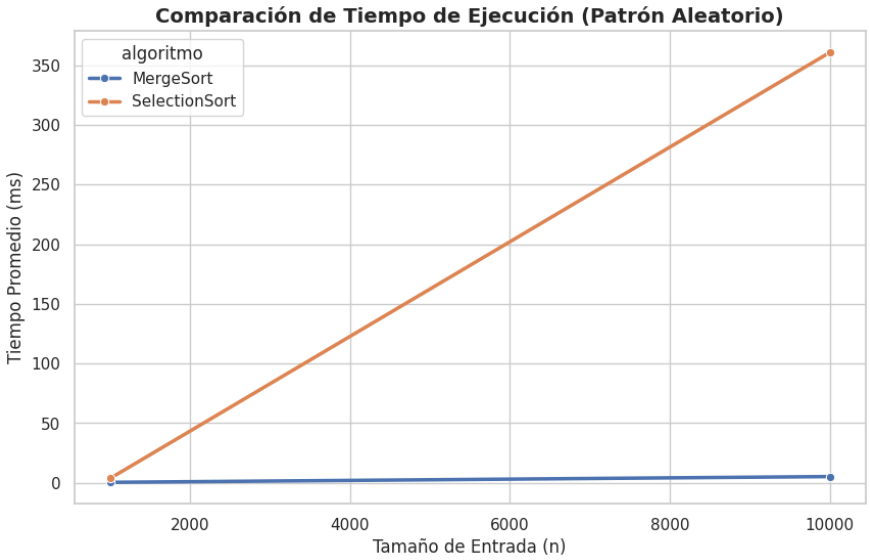


Figura 2: Comparación de tiempos de ejecución (ms) según el patrón de datos.

Por otro lado, la Figura 3 muestra la distribución y dispersión de las 30 repeticiones mediante diagramas de caja, permitiendo identificar la presencia de valores atípicos (outliers) y la notable variabilidad en las ejecuciones de mayor escala para el algoritmo cuadrático.

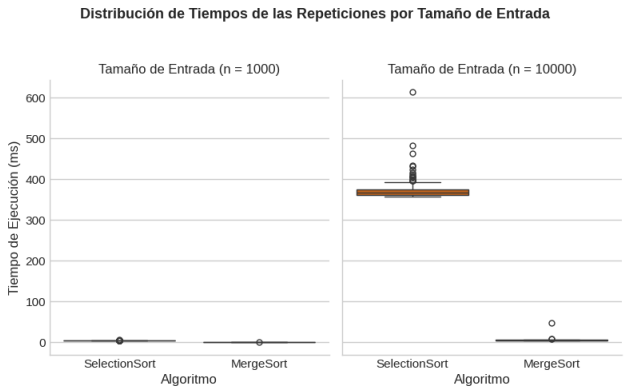


Figura 3: Comparacion de la distribución y dispersión de las 30 repeticiones

4. Discusión

Los hallazgos respaldan la teoría de que Selection Sort no es un algoritmo adaptativo. La limitación principal del estudio radicó en la escala máxima evaluada, restringida a $n = 10,000$ debido a la saturación de CPU. Las amenazas a la validez interna se controlaron promediando las 30 ejecuciones independientes para mitigar el ruido del sistema operativo. Por otra parte, MergeSort demostró un rendimiento sumamente eficiente y con-

sistente, validando empíricamente su complejidad teórica de $O(n \log n)$. A diferencia de Selection Sort, MergeSort mantuvo tiempos de ejecución inferiores a los 6 milisegundos incluso para $n = 10,000$, lo que confirma su alta escalabilidad y su idoneidad para procesar volúmenes de datos significativamente mayores. Asimismo, el análisis de estabilidad permitió corroborar que MergeSort preserva el orden relativo de los elementos con claves duplicadas, un factor determinante frente al comportamiento inestable y rígido observado en Selection Sort.

5. Conclusiones

1. Comprobación empírica de Selection Sort: Se demostró experimentalmente que el número de comparaciones en Selection Sort cumple de forma estricta la cota teórica $C(n) = \frac{n(n-1)}{2}$, evidenciando un crecimiento cuadrático invariable ante los distintos patrones de entrada.

2. Rendimiento superior de MergeSort: Se validó que MergeSort supera drásticamente en eficiencia a Selection Sort, manteniendo tiempos de ejecución mínimos y estables de $O(n \log n)$, lo que lo posiciona como un enfoque óptimo para grandes volúmenes de datos.

3. Impacto de la estabilidad y patrones de datos: Se confirmó que Selection Sort carece de adaptabilidad frente a datos parcialmente ordenados, mientras que MergeSort demostró un comportamiento predecible y seguro al preservar la estabilidad ante claves duplicadas.

4. Rigurosidad metodológica: La estrategia de promediar 30 repeticiones independientes utilizando semillas fijas otorgó alta solidez estadística a las métricas recolectadas, permitiendo mitigar el ruido del sistema operativo.

5. Justificación de escalabilidad: La omisión formal de la prueba con $n = 100,000$ quedó técnicamente justificada por la inviabilidad temporal derivada de la complejidad $O(n^2)$ en el algoritmo iterativo.

Agradecimientos

Se agradece al docente de la asignatura de Programación Avanzada por las pautas metodológicas facilitadas para el desarrollo de esta investigación.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

- Knuth, D. E. (1998). *The Art of Computer Programming: Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.